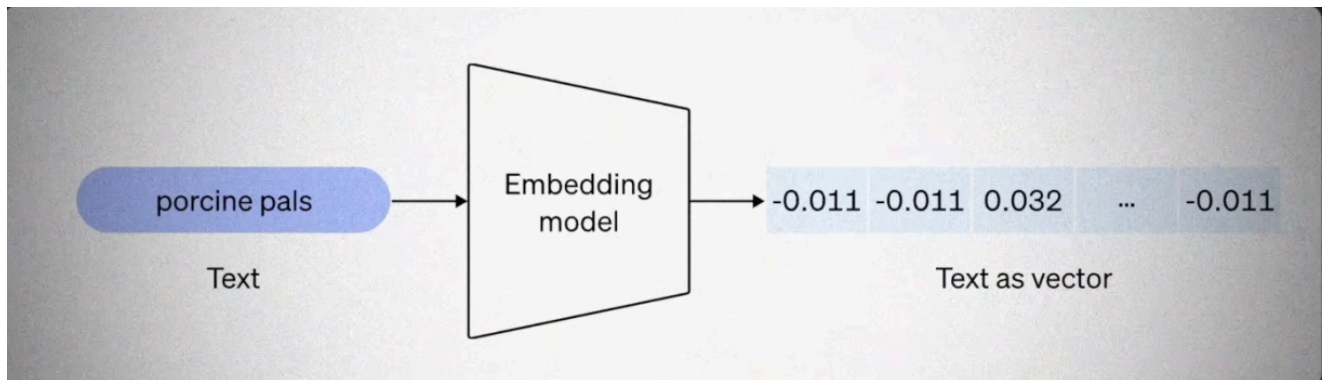


Embedding v2

Embedding turns the input data into high-dimensional dense vectors that capture the meaning of each token.

For example: The word "cat" has an embedding vector closer to similar words like "dog" or "cat" than to "rat"

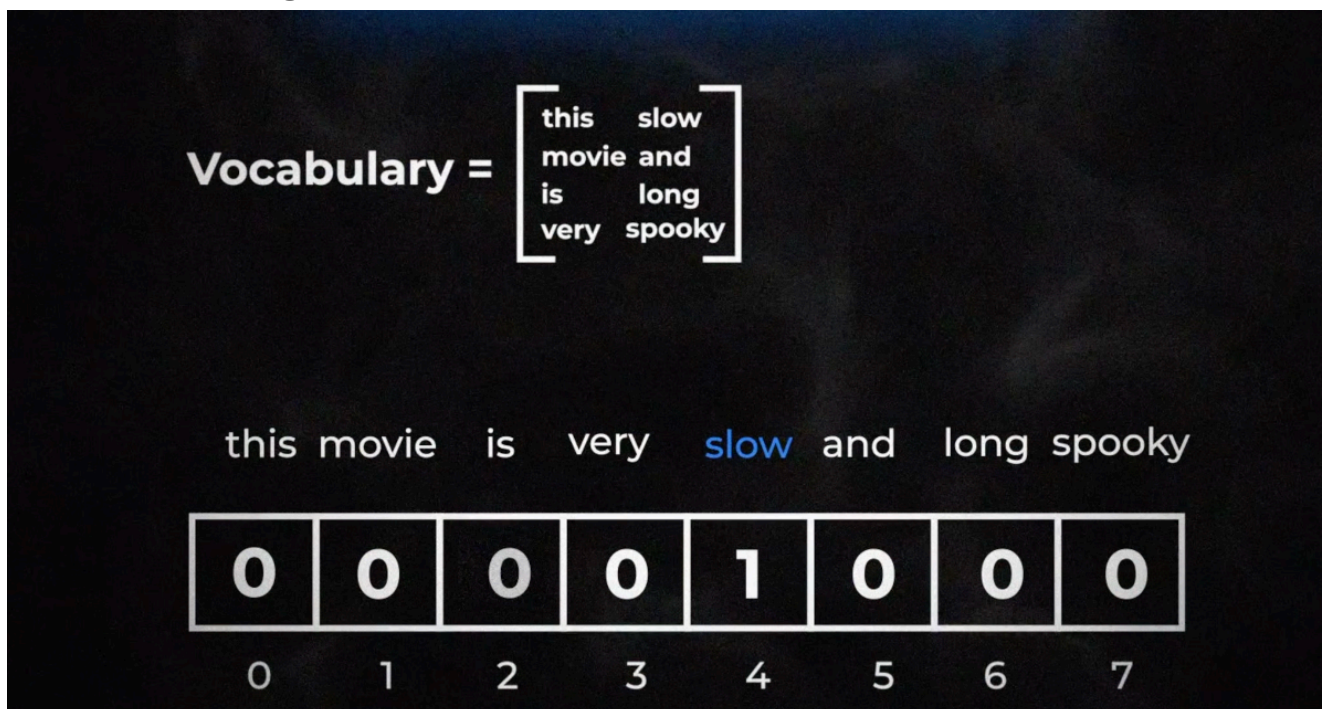


Text Embedding falls into 2 categories:

- **Word/Token embedding:** Represents a single word or token.
- **Document embedding:** Represent the whole document or sentence/paragraph.

There also simpler and conventional approach such as

One-hot Encoding



Bag of word

Vocabulary =

this	slow
movie	and
is	long
very	spooky

"Long and spooky movie"

this movie is very slow and long spooky

0	1	0	0	1	0	1	1
---	---	---	---	---	---	---	---

N-gram (2-gram) is similar to the Bag of word

TF-IDF (Term Frequency Inverse Document Frequency)

$$TF = \frac{\text{Number of times this word occurred}}{\text{Number of words in document}}$$

$$IDF = \frac{\text{Number of document}}{\text{Number of documents where this words occurred}}$$

These approaches fall under the "Frequency-Based Representation Techniques"

- They do not take context into account (I am fine vs I got a fine)
- Not take care of the order of the word in the sentence (My cat bite my dog vs < My dog bite my cat.)
- They do not capture the underlying meaning of the words (Meal was bad and Food is awful)

"Two words that occur in very similar distributions have similar meaning"

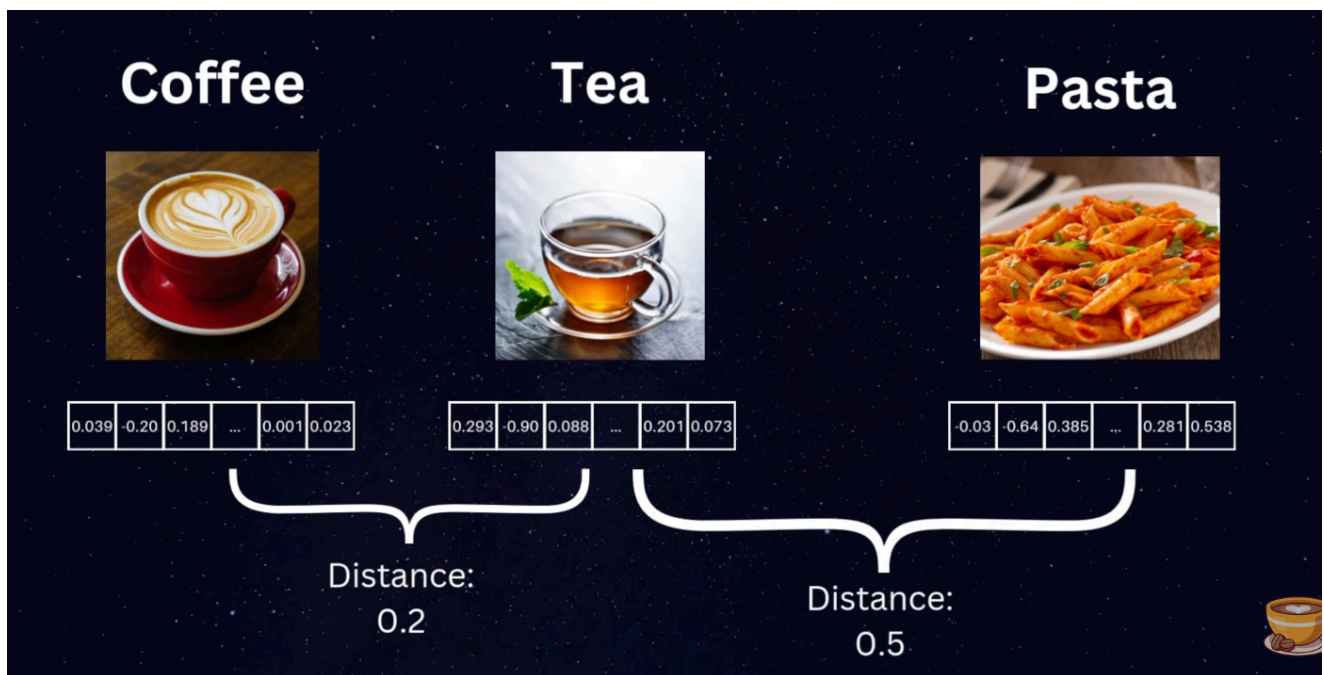
Joos (1950), Harris (1954), and Firth (1957)

Dense Vector = a mathematical representation of data (like text, images, or audio) where most or all elements hold non-zero values, organized into fixed-length arrays. So these numeric array act as "embeddings" that capture the semantic meaning of your data, grouping related concepts close together in a high-dimensional mathematical space.

An embedding of a word has fewer dimensions than the number of word that you have in your vocabulary.

The size of the embedding vector depend on the embedding models, but often range between 50 and a few thousands

Word that has similar meaning stay close together in the vector space.

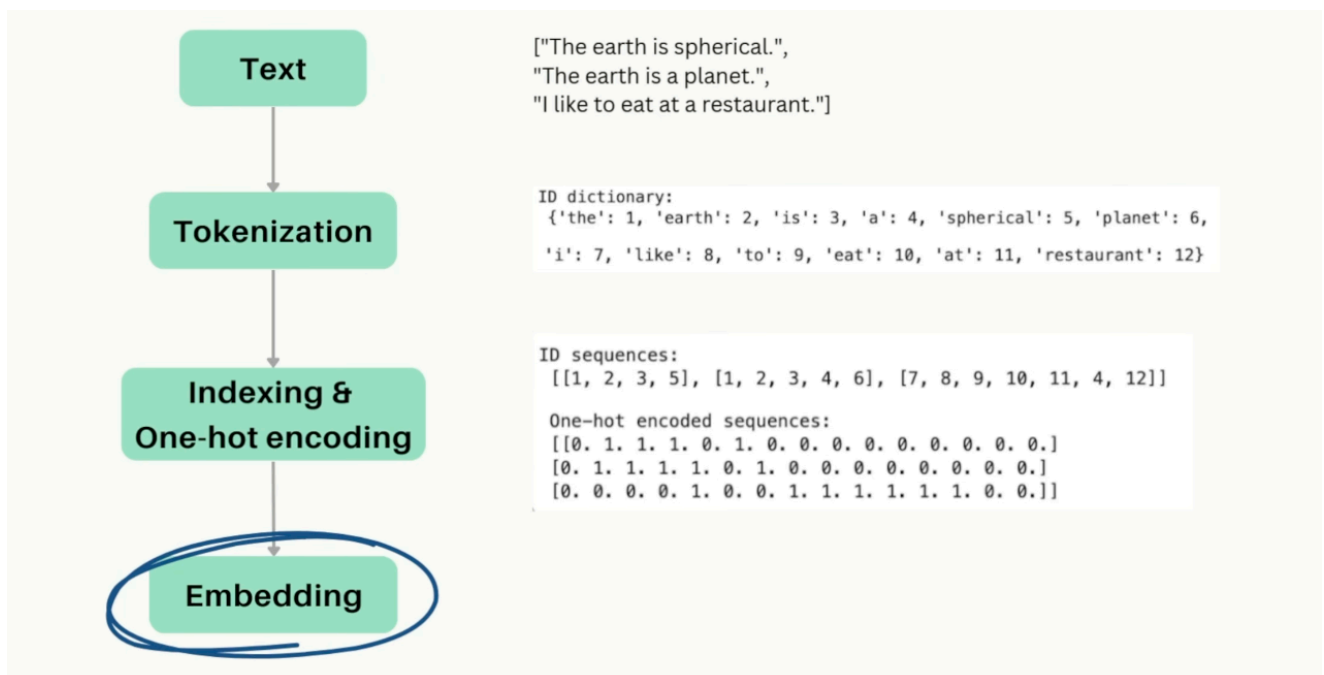


Embedding space = a mathematical representation where data are translated into sequences of numbers called vectors.

From 2 dimension upward, we have to use the **Euclidean distance** or **cosine similarity** to calculate the distance.

And yes, if we are working with big dimension space, we can use [Dimensionality reduction](#)

Cycle of Embedding

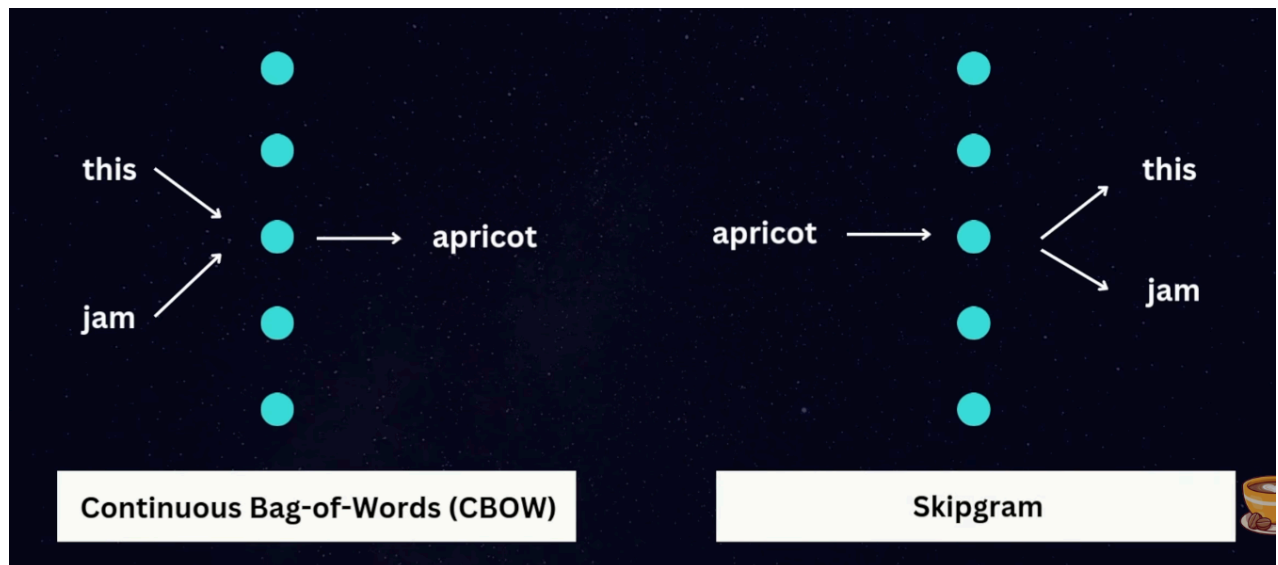


One of the most popular embedding models is **Word2Vec**: By getting the One-hot encoding version of the words, then creating the embedding of these words by using the context of the words.

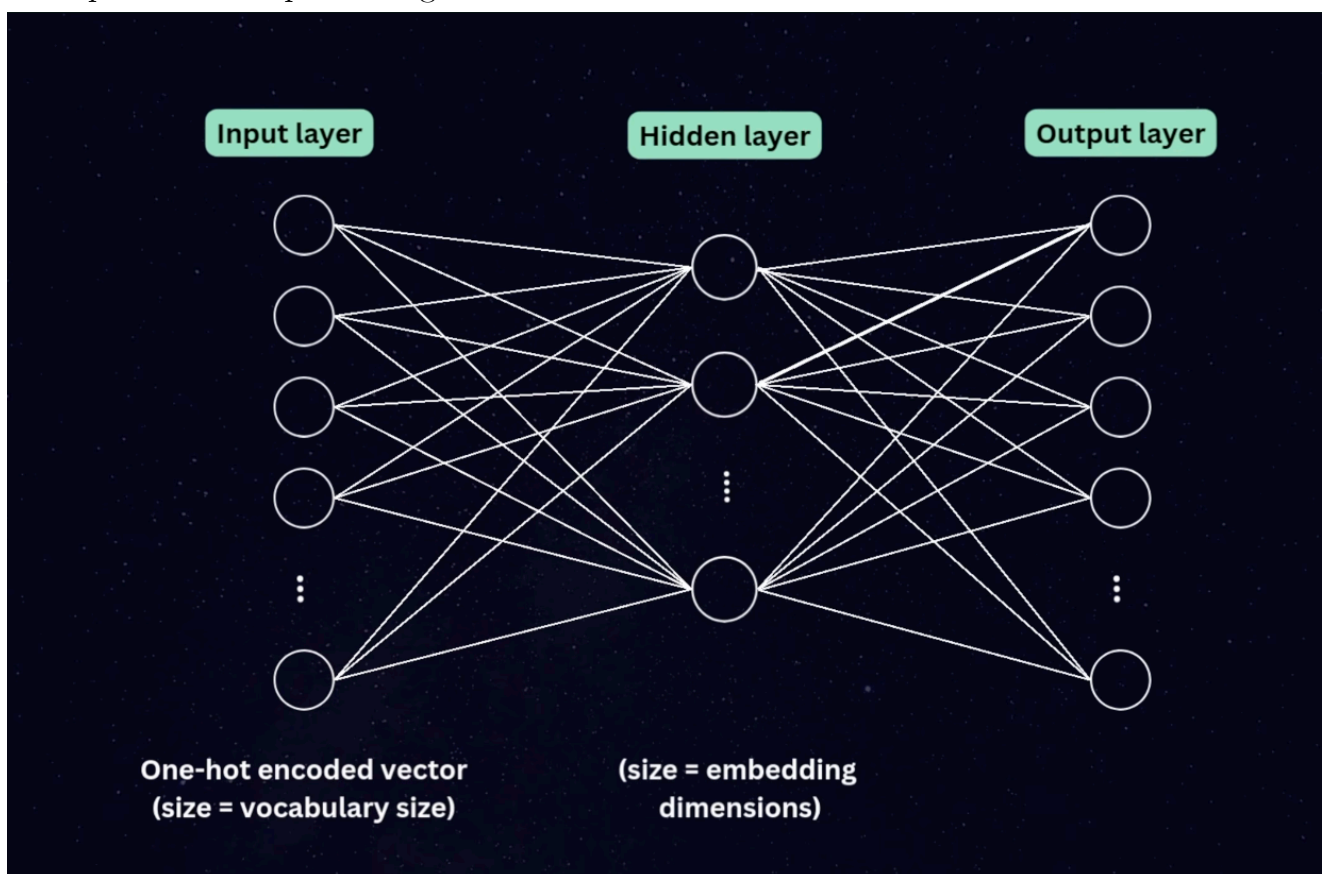
Word2Vec:

- Continuous Bag of word (CBOW)

- Skip gram



Example in the Deep Learning



The number of neurons in the hidden layer will be the size of the embedding that we will get
 The number of neurons in the output layer will be the size of the total vocabulary in the corpus, and the same thing for the input layer.

Text embedding methods

Contextual Embedding and Static Embedding

Text embedding methods

Static embedding
(Word2Vec, GloVe, etc.)

Contextual embedding
(ELMO, BERT, GPT)

The contextual embedding, the embedding of the same word, may be different in different contexts. So it lands a big leap forward than the static embedding.

For the transformer architecture, the embedding doesn't just embed the words, they process entire sequences of text and refines embeddings by considering the context of the word in the sequence, and all of these are done by using [Self-attention](#) which allows the model to weight the importance of all the words in the sentence when creating the embedding.

Credit to Thu Vu

Using OpenAI embedding API

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")

response = client.embeddings.create(
    model="text-embedding-3-small",
    input="Machine learning is transforming agriculture."
)

embedding = response.data[0].embedding

print(f"Embedding dimension: {len(embedding)}")
print(embedding[:10]) # first 10 values
```

Cosine Similarity

```
from openai import OpenAI
import numpy as np

client = OpenAI(api_key="YOUR_API_KEY")

texts = [
```

```

    "AI can help farmers detect pests.",
    "Machine learning can identify crop diseases."
]

response = client.embeddings.create(
    model="text-embedding-3-small",
    input=texts
)

emb1 = np.array(response.data[0].embedding)
emb2 = np.array(response.data[1].embedding)

cosine_similarity = np.dot(emb1, emb2) / (
    np.linalg.norm(emb1) * np.linalg.norm(emb2)
)

print(f"Similarity: {cosine_similarity:.4f}")

```

LangChain

```

from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

vector = embeddings.embed_query(
    "How can AI improve crop yield?"
)

print(len(vector))

```

Multiple Documents

```

docs = [
    "Rice farming techniques",
    "Pest detection using AI",
    "Climate-smart agriculture"
]

vectors = embeddings.embed_documents(docs)

print(len(vectors))
print(len(vectors[0]))

```

Store the Embedding in the vector database

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

documents = [
    "AI helps detect crop diseases.",
    "Smart irrigation saves water.",
    "Satellite imagery monitors farms."
]

embedding_model = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

vector_store = FAISS.from_texts(
    documents,
    embedding_model
)

results = vector_store.similarity_search(
    "How can technology monitor farms?",
    k=2
)

for doc in results:
    print(doc.page_content)
```